

Analyse et traitement des données

Des données brutes aux décisions, avec Python

Issa Gueye

Doctorant en mathématiques appliquées · Ingénieur calcul scientifique

Plan du cours

#	Chapitre
0	Introduction, cycle CRISP-DM, types de variables, qualité
1	Python scientifique et NumPy
2	pandas : les fondamentaux (pandas 3.0)
3	Importer, exporter, auditer la qualité
4	Nettoyage : types, texte, doublons, manquants, aberrants
5	Transformer : groupby, jointures, pivots, séries temporelles
6	Analyse exploratoire et visualisation
7	Statistique inférentielle
8	Feature engineering et pipelines
9	ACP, clustering, introduction à la modélisation

Pourquoi ce cours insiste sur le *traitement*

- Dans un projet réel, **la majorité du temps** part dans la collecte, le nettoyage et la mise en forme.
- Un modèle sophistiqué sur des données sales donne des **conclusions fausses** avec beaucoup d'assurance.
- *Garbage in, garbage out.*

Fil rouge : des données **simulées** inspirées du Sénégal — ventes de boutiques, météo de Dakar, enquête ménages, résultats d'étudiants, clients mobile money.

Le cycle CRISP-DM

1. **Compréhension métier** — quelle question ? quel critère de succès ?
2. **Compréhension des données** — sources, qualité, biais
3. **Préparation** — nettoyage, transformation, variables
4. **Modélisation**
5. **Évaluation** — répond-on à la question ?
6. **Déploiement** — rapport, tableau de bord, automatisation

➡ **Processus itératif** : on revient souvent en arrière.

Types de variables

Type	Exemple	Résumé	Graphique
Nominale	région, mode de paiement	effectifs, mode	barres
Ordinale	niveau d'éducation	médiane, quantiles	barres ordonnées
Discrète	taille du ménage	moyenne, médiane	histogramme
Continue	revenu, température	moyenne, médiane, quantiles	histogramme, boîte

⚠ Un **code** numérique (identifiant, « 99 = ne sait pas ») n'est pas une quantité.

Tidy data (Wickham, 2014)

1. Chaque **variable** est une **colonne**
2. Chaque **observation** est une **ligne**
3. Chaque **type d'unité** forme une **table**

C'est le format attendu par pandas, seaborn et scikit-learn.

`melt` (large → long) et `pivot` (long → large) permettent d'y arriver.

Dimensions de la qualité

Dimension	Défaut typique
Complétude	revenu non déclaré
Validité	quantité négative, température -99 °C
Exactitude	999 saisi au lieu de 9
Cohérence	« Dakar », « dakar », « DAKAR »
Unicité	transaction importée deux fois
Actualité	référentiel obsolète

NumPy : vectoriser

```
prix = np.array([12500, 6500, 700, 2300])
qte  = np.array([[3, 2, 10, 4],
                 [1, 0, 25, 2]])          # 2 boutiques × 4 produits
ca = qte * prix                          # broadcasting (2,4) * (4,)
ca.sum(axis=1)                           # CA par boutique
Z = (X - X.mean(axis=0)) / X.std(axis=0, ddof=1)
```

- Opérations en C, sans boucle Python : **10 à 100× plus rapide**
- Broadcasting : comparer les formes **de droite à gauche** (égales ou 1)
- `rng = np.random.default_rng(42)` : aléatoire **reproductible**

pandas : sélectionner

	Sélection par	Fin
<code>df.loc[...]</code>	étiquettes / booléens	incluse
<code>df.iloc[...]</code>	positions	exclue

```
df[(df["milieu"] == "Rural") & (df["taille"] > 15)]  
df.query("milieu == 'Rural' and taille > 15")  
df.loc[df["taille"] == 99, "taille"] = np.nan      # modifier : toujours .loc
```

Nouveautés pandas 3.0

- **Type** `str` par défaut pour le texte (plus rapide, plus sûr)
- **Copy-on-Write** : toute sélection se comporte comme une copie
 - `df["a"][masque] = 0` ❌ n'a plus d'effet
 - `df.loc[masque, "a"] = 0` ✅
- `df.assign(z=pd.col("x") / pd.col("y"))`
- Dates en microsecondes par défaut

Conseil de migration : passer d'abord par pandas 2.3 sans avertissement.

Lire correctement un CSV

```
pd.read_csv("f.csv",  
            sep=";", decimal=".", thousands=" ",  
            encoding="latin-1",  
            na_values=["NSP", "-"],  
            dtype={"code_postal": str},  
            parse_dates=["date"], dayfirst=True)
```

Formats : **CSV** (échange) · **Excel** (non-programmeurs) · **JSON** (API) · **Parquet** (stockage analytique, typé, compact)

Auditer avant d'analyser

```
profil(df)                                # type, % manquants, n uniques, exemple
df.duplicated().sum()
df["region"].value_counts()                # 9 régions écrites de 30 façons !
regles = {"quantité > 0": (df.quantite > 0).all(), ...}
```

Écrire des **règles de validation** testables, les relancer après nettoyage.

Nettoyer : les outils de base

```
pd.to_numeric(s.str.replace(r"[^\d,.\-]", "", regex=True), errors="coerce")
pd.to_datetime(s, format="%d/%m/%Y", errors="coerce")
s.str.strip().str.title().replace({"St-Louis": "Saint-Louis"})
df.drop_duplicates()
```

- **Compter** ce que chaque étape change (journal)
- Vérifier contre un **référentiel**
- Encapsuler dans une **fonction ré-exécutable**

Données manquantes : les mécanismes (Rubin, 1976)

	Dépend de...	Exemple
MCAR	rien	capteur en panne au hasard
MAR	variables observées	les urbains diplômés déclarent moins leur revenu
MNAR	la valeur elle-même	les plus riches cachent leur revenu

➡ Explorer : taux de manquants **par groupe**.

Imputer : que montre la simulation ?

Méthode	Moyenne	Écart-type	Erreur
Suppression	biaisée	—	—
Moyenne globale	biaisée	écrasé	forte
Médiane par groupe	mieux	mieux	moyenne
KNN / MICE	≈ vérité	≈ vérité	faible

En modélisation : l'imputeur est ajusté sur le **train** uniquement.

Valeurs aberrantes

- **Tukey** : hors de $[Q_1 - 1,5 IQR, Q_3 + 1,5 IQR]$
- **z robuste** : $|x - \text{mé d}| / (1,4826 \cdot MAD) > 3,5$
- Revenu log-normal : 155 « aberrants » en échelle brute, **15** en échelle log

La statistique **détecte**, le **métier décide** : corriger, garder, winsoriser — jamais supprimer en silence.

Transformer : split – apply – combine

```
df.groupby("region").agg(n=("id", "count"), ca=("montant", "sum"))
df["part"] = df["montant"] / df.groupby("region")["montant"].transform("sum")
pd.pivot_table(df, values="montant", index="categorie", columns="paiement", aggfunc="sum")
a.merge(b, on="region", how="left", validate="many_to_one", indicator=True)
serie.resample("ME").sum(); serie.rolling(7).mean(); serie.pct_change()
```

⚠ Jointures : vérifier l'unicité des clés et le nombre de lignes.

Analyse exploratoire : quel graphique ?

Variables	Graphique
1 quanti	histogramme, ECDF, boîte
1 quali	barres triées
quanti × quanti	nuage (log), hexbin
quanti × quali	boîtes, violons, points + IC
quali × quali	barres empilées à 100 %
temps	courbe + moyenne mobile

Toujours tracer : le quartet d'Anscombe a les mêmes moyennes, variances et corrélations... et 4 formes différentes.

Un bon graphique

1. Le **titre** énonce le message
2. Axes étiquetés, **unités**, échelle log signalée
3. Barres qui commencent à **zéro**
4. Pas de 3D ni de décoration inutile
5. Couleurs **accessibles** ; l'information n'est pas codée par la couleur seule
6. Catégories **triées**
7. Montrer l'**incertitude**

Inférence : estimer et tester

- **IC 95 %** : $\bar{x} \pm t_{0,975;n-1} s / \sqrt{n}$
95 % des intervalles construits ainsi contiennent la vraie valeur
- **p-valeur** : probabilité, *si H_0 est vraie*, d'un résultat au moins aussi extrême
- Toujours rapporter **IC + taille d'effet** (d de Cohen, η^2 , V de Cramér)
- **Association $\neq$ causalité**

Quel test ?

Situation	Test	Alternative
2 groupes	t de Welch	Mann–Whitney
Apparié	t apparié	Wilcoxon signé
≥ 3 groupes	ANOVA + Tukey	Kruskal–Wallis
2 qualitatives	χ^2	Fisher exact
2 quantitatives	Pearson	Spearman
Statistique quelconque	bootstrap	

20 tests à 5 % $\rightarrow$ 64 % de chance d'un faux positif : **Holm / Benjamini–Hochberg**.

Feature engineering

- **Encoder** : one-hot (nominal), ordinal (ordinal)
- **Échelle** : `StandardScaler`, `RobustScaler` ; **forme** : log, Yeo-Johnson
- **Créer** : ratios, dates, discrétisation, interactions, indicateurs de manquant
- Variables cycliques : $\sin(2\pi m/12)$, $\cos(2\pi m/12)$

La fuite de données

Sélectionner 20 variables parmi 5 000 variables **de bruit** *avant* la validation croisée :

	Exactitude
Avec fuite	0,85 🙌
Sans fuite (dans le pipeline)	0,48 (hasard)

➡ Tout ce qui **apprend** des données va **dans le** `Pipeline` .

Pipeline scikit-learn

```
prep = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median"), StandardScaler()), num),
    ("nom", make_pipeline(SimpleImputer(strategy="most_frequent"),
                          OneHotEncoder(handle_unknown="ignore")), cat),
])
modele = make_pipeline(prepare, LogisticRegression())
cross_val_score(modele, X_train, y_train, cv=5, scoring="roc_auc")
```

Un seul objet : pas de fuite, sauvegardable, applicable aux données brutes.

ACP

- Standardiser, puis diagonaliser la matrice de corrélation : $R = V\Lambda V^\top$
- $\lambda_k / \sum \lambda_j$ = part de variance de la composante k
- Choisir le nombre d'axes : éboulis, Kaiser ($\lambda > 1$), % cumulé
- **Cercle des corrélations** pour interpréter les axes

Clustering

- **k-means** : minimise l'inertie intra ; standardiser ; `n_init=10`
- Choisir k : coude, **silhouette**, **interprétabilité**
- Dans notre exemple, la silhouette propose k = 3 alors que 4 profils existent : les critères **aident**, ils ne décident pas
- **Profiler et nommer** les segments : c'est là que se crée la valeur métier
- Alternatives : CAH Ward, DBSCAN, mélanges gaussiens

Modéliser avec rigueur

1. Séparer **train / test** (le test est mis sous clé)
2. Comparer à une **baseline** naïve
3. Choisir par **validation croisée**
4. **Une seule** évaluation finale sur le test
5. Métriques adaptées : MAE/RMSE/ R^2 · précision/rappel/F1/AUC

Ressources du dépôt

- `cours/` : 10 notebooks exécutés
- `exercices/` : 4 TD (énoncés + corrigés)
- `projets/` : 3 projets intégrateurs avec grille d'évaluation
- `memos/` : fiches pandas, nettoyage, stats/visualisation/ML
- `data/` : 6 jeux simulés + script de génération
- `references/` : bibliographie commentée

Merci !

Issa Gueye

github.com/iamnewton4596/analyse-traitement-donnees